Universidad Politécnica de Atlautla

Actividad 2.3. Búsqueda local

"Implementación y optimización con Hill Climbing y Simulated Annealing"

Alumno: Ronaldo Malcom Castro Soriano

Docente: Ing. Javier Garcia Lara.

22 de Octubre de 2026

1. Tema y planteamiento del problema

Se plantea optimizar la ruta de un pequeño reparto de artesanías. Un vehículo parte del Depósito/Taller, visita una vez cada uno de los puntos de entrega o venta y regresa al depósito. El objetivo es reducir la distancia total recorrida. Este ejemplo corresponde a una variante del problema del viajante y permite comparar dos técnicas de búsqueda local, que exploran soluciones vecinas en lugar de construir y conservar todo el árbol de búsqueda (Russell & Norvig, 2021; Wirsansky, 2020).

Las ubicaciones se representan sobre un plano artificial. Las coordenadas, longitudes y costos de este reporte son datos simulados para la práctica: no son ubicaciones geográficas ni distancias oficiales de municipios reales.

2. Datos del problema

Se utilizan nueve puntos: un depósito fijo y ocho destinos. La distancia entre dos puntos se calcula mediante distancia euclidiana. Para la ruta se suma el costo de cada tramo desde el depósito hasta cada destino y finalmente de regreso al depósito.

Ubicación	Coordenada x	Coordenada y
Depósito/Taller	11	9
Artesano A	12	4
Artesano B	16	16
Artesano C	13	3
Artesano D	10	7
Artesano E	5	19
Punto de venta	20	1
Centro de acopio	17	12
Mercado local	8	6

3. Formulación del problema

Estado: una permutación de los ocho destinos; el depósito se mantiene como origen y destino final. **Acciones:** aplicar un movimiento 2-opt, que invierte el orden de un segmento de la ruta. **Vecindad:** conjunto de rutas que se pueden obtener con una sola inversión 2-opt. **Función objetivo:** minimizar la suma de distancias de todos los tramos. **Estado inicial:** una permutación aleatoria reproducible con semilla 1010. **Meta:** encontrar una ruta de menor costo; Hill Climbing se detiene cuando no encuentra mejores vecinos y Simulated Annealing después de 5,000 iteraciones.

La distancia entre i y j se calcula como d(i,j) = raíz cuadrada de $((x_i - x_j)^2 + (y_i - y_j)^2)$. La función de costo de una ruta R es f(R) = suma de $d(R_k, R_{k+1})$, incluyendo el trayecto de ida desde el depósito y el regreso final.

4. Algoritmos y parámetros

Hill Climbing. En cada ronda se generan todos los vecinos 2-opt y se elige el de menor costo, siempre que mejore el costo actual. Si ningún vecino mejora la solución, se detiene. Es sencillo y rápido, pero puede terminar en un óptimo local.

Simulated Annealing. En cada iteración elige un vecino 2-opt al azar. Si el vecino mejora la ruta, se acepta; si empeora, puede aceptarse con probabilidad $\exp(-\Delta/T)$. La temperatura disminuye mediante $T(t) = T_0 \cdot \eta^t$. El mecanismo permite aceptar temporalmente movimientos peores y escapar de ciertos óptimos locales (Kirkpatrick et al., 1983; Russell & Norvig, 2021).

Parámetro	Hill Climbing	Simulated Annealing
Temperatura inicial T_0	No aplica	12.0

Parámetro	Hill Climbing	Simulated Annealing
Factor de enfriamiento η	No aplica	0.997
Iteraciones/rondas máximas	100 rondas; termina antes si no mejora	5,000 iteraciones
Vecino	Mejor vecino 2-opt	Vecino 2-opt aleatorio
Semilla aleatoria	1010 para estado inicial	2026 para decisiones aleatorias

5. Implementación en Python

El archivo actividad_2_3_busqueda_local.py construye la matriz de distancias, genera el estado inicial, ejecuta ambos algoritmos, contabiliza evaluaciones y tiempo, produce las gráficas y guarda los resultados en JSON. NetworkX se usa para dibujar las rutas y Matplotlib para las gráficas. La medición de tiempo utiliza un contador de alta resolución de Python.

Para ejecutar el programa se requiere Python 3 y las bibliotecas matplotlib y networkx. Comandos: python -m pip install matplotlib networkx; después, python actividad_2_3_busqueda_local.py.

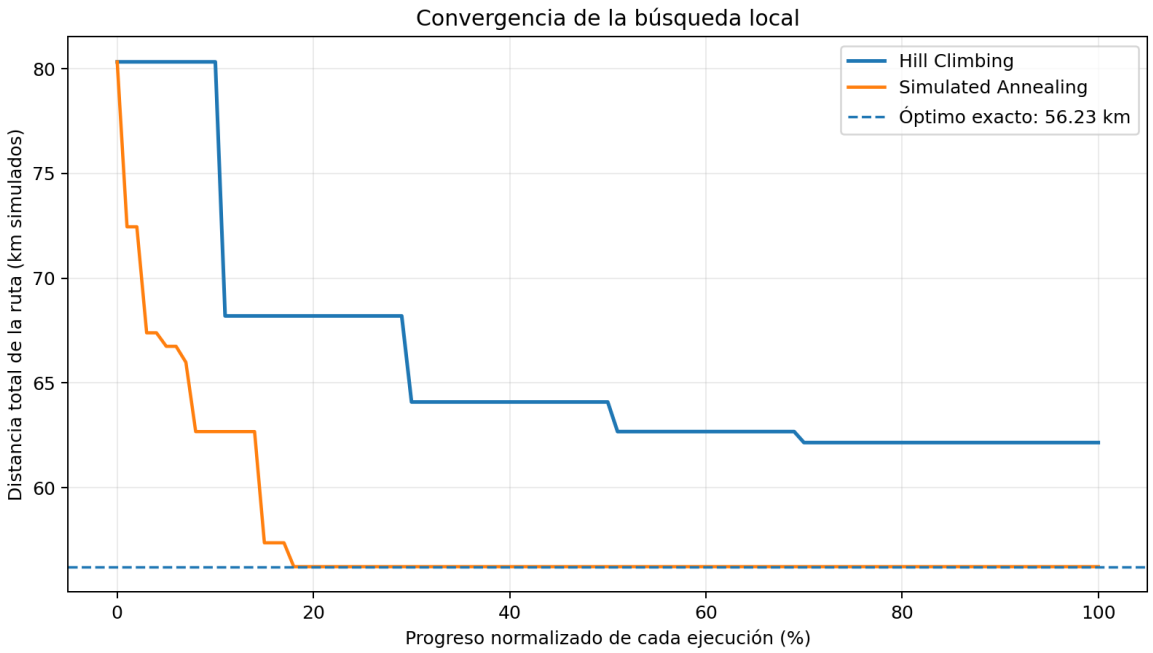
6. Resultados y comparación

Método	Distancia final	Evaluaciones	Rondas/iteraciones	Tiempo medido
Ruta inicial	80.33 km	—	—	—
Hill Climbing	62.15 km	140	5	0.18 ms
Simulated Annealing	56.23 km	5000	5000	14.51 ms
Óptimo exacto de referencia	56.23 km	40320	Enumeración completa	No comparable

La ruta inicial cuesta 80.33 km simulados. Hill Climbing reduce el costo a 62.15 km, una mejora de 22.6%. Simulated Annealing llega a 56.23 km, una mejora de 30.0%, que coincide con el óptimo obtenido al enumerar las 40,320 permutaciones posibles. En esta prueba, Hill Climbing necesitó menos evaluaciones y tiempo, pero quedó atrapado en una solución local de mayor costo. Simulated Annealing requirió más evaluaciones y alcanzó la mejor ruta encontrada por la enumeración.

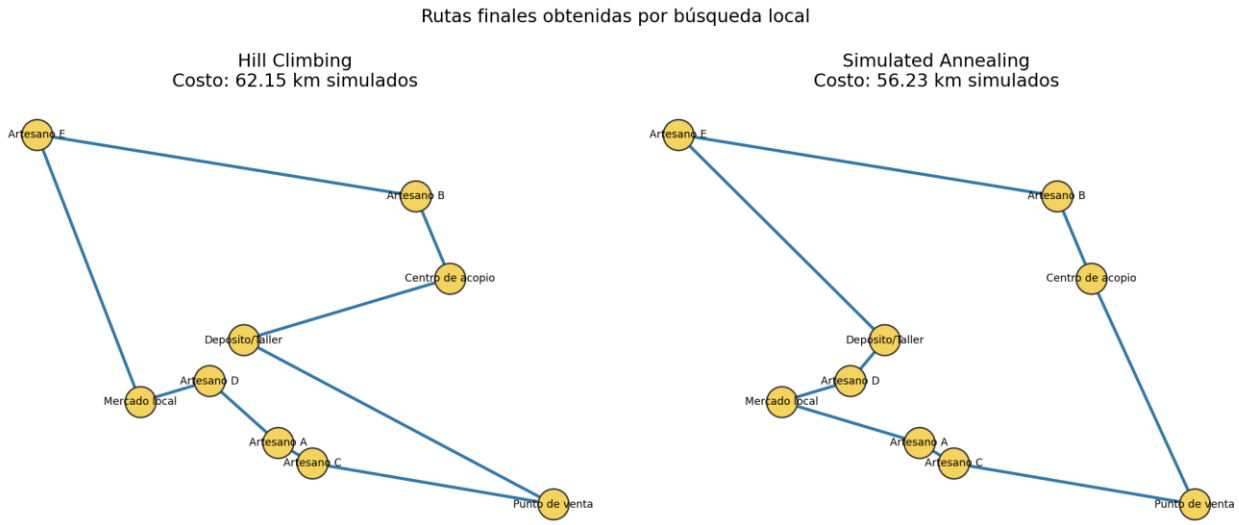
Los tiempos de ejecución solo describen esta ejecución y dependen del equipo, la carga del sistema y el intérprete de Python. No deben interpretarse como una comparación universal del rendimiento.

6.1. Gráfica de convergencia



La gráfica muestra el mejor costo encontrado a medida que avanza cada método. El eje horizontal representa el progreso normalizado de cada ejecución, porque los algoritmos recorren cantidades diferentes de iteraciones. La línea punteada corresponde al costo de la ruta óptima exacta.

6.2. Rutas finales



La representación utiliza las coordenadas artificiales del ejercicio y no es un mapa real. La mejor ruta de Simulated Annealing y la obtenida por enumeración coinciden; la ruta de Hill Climbing conserva una longitud mayor.

7. Análisis de complejidad

Sea n el número de destinos sin contar el depósito. En la implementación de Hill Climbing se evalúan $O(n^2)$ vecinos por ronda; cada evaluación recalcula el costo de la ruta en O(n). Por ello, si se ejecutan R rondas, el tiempo de este procedimiento es $O(R \cdot n^3)$. La memoria adicional del método es O(n + R), porque conserva la ruta actual y un historial de costos.

Simulated Annealing evalúa un solo vecino por iteración. Como el programa recalcula la ruta en O(n) para cada vecino, el tiempo de I iteraciones es $O(I \cdot n)$. La memoria es O(n + I), dado que se guarda el historial de mejor costo para dibujar la convergencia. La matriz de distancias ocupa $O(n^2)$ memoria.

El programa también ejecuta una enumeración exacta para contar todas las permutaciones posibles. Esta comprobación tiene costo $O(n! \cdot n)$ y se incluye solo como referencia para evaluar la calidad de las heurísticas; no forma parte de la complejidad de Hill Climbing ni de Simulated Annealing.

8. Conclusiones

En el ejemplo, Hill Climbing encontró con rapidez una mejora importante, pero se detuvo en un óptimo local. Simulated Annealing exploró más estados, aceptó algunos movimientos peores durante la fase de temperatura alta y encontró la ruta de menor costo verificada por enumeración exacta. Por tanto, Hill Climbing puede convenir cuando se desea rapidez y el costo de una solución local es aceptable; Simulated Annealing es más apropiado cuando se dispone de más tiempo de cálculo y se busca reducir el riesgo de quedarse en una solución local. El resultado depende de la vecindad, la solución inicial, la temperatura y el calendario de enfriamiento.

9. Referencias (APA 7.ª edición)

Kirkpatrick, S., Gelatt, C. D., Jr., & Vecchi, M. P. (1983). Optimization by simulated annealing. *Science, 220*(4598), 671–680. https://doi.org/10.1126/science.220.4598.671

NetworkX Developers. (s. f.). *Drawing*. NetworkX documentation. https://networkx.org/documentation/stable/reference/drawing.html

Python Software Foundation. (s. f.). *time — Time access and conversions*. Python documentation. https://docs.python.org/3/library/time.html

Russell, S. J., & Norvig, P. (2021). *Artificial intelligence: A modern approach* (4th ed.). Pearson.

Wirsansky, E. (2020). *Hands-on genetic algorithms with Python: Applying genetic algorithms to solve real-world deep learning and artificial intelligence problems*. Packt Publishing.

Anexo A. Código Python completo: actividad 2.3

El siguiente anexo reproduce el archivo Python incluido por separado. El código está como texto editable; para ejecutarlo se recomienda usar el archivo .py original.

```
"""Actividad 2.3. Búsqueda local: Hill Climbing y Simulated Annealing.

Caso didáctico: optimización de una ruta de reparto de artesanías.
Las coordenadas y distancias son simuladas; no representan mediciones oficiales.

Instalación:
    python -m pip install matplotlib networkx

Ejecución:
    python actividad_2_3_busqueda_local.py
"""
from __future__ import annotations

import itertools
import json
import math
import random
import time
from pathlib import Path

import matplotlib.pyplot as plt
import networkx as nx

BASE_DIR = Path(__file__).resolve().parent
FIG_DIR = BASE_DIR / "figuras"
FIG_DIR.mkdir(parents=True, exist_ok=True)

# Coordenadas didácticas en un plano expresado en kilómetros aproximados.
# La primera ubicación es el depósito y siempre será el inicio y el final.
UBICACIONES = {
    "Depósito/Taller": (11.0, 9.0),
    "Artesano A": (12.0, 4.0),
    "Artesano B": (16.0, 16.0),
    "Artesano C": (13.0, 3.0),
    "Artesano D": (10.0, 7.0),
    "Artesano E": (5.0, 19.0),
    "Punto de venta": (20.0, 1.0),
    "Centro de acopio": (17.0, 12.0),
    "Mercado local": (8.0, 6.0),
}
NOMBRES = list(UBICACIONES)
COORDENADAS = [UBICACIONES[nombre] for nombre in NOMBRES]
N = len(NOMBRES)
DEPOSITO = 0



def crear_matriz_distancias(coordenadas: list[tuple[float, float]]) -> list[list[float]]:
    """Calcula distancias euclidianas entre todos los puntos simulados."""
    matriz = []
    for x1, y1 in coordenadas:
        fila = []
        for x2, y2 in coordenadas:
            fila.append(math.hypot(x2 - x1, y2 - y1))
        matriz.append(fila)
    return matriz



DISTANCIAS = crear_matriz_distancias(COORDENADAS)



def costo_ruta(ruta: list[int]) -> float:
    """Suma los tramos desde el depósito, pasando por la ruta y regresando."""
    recorrido = [DEPOSITO] + ruta + [DEPOSITO]
    return sum(DISTANCIAS[recorrido[i]][recorrido[i + 1]]
               for i in range(len(recorrido) - 1))



def vecino_2opt(ruta: list[int], i: int, j: int) -> list[int]:
    """Genera un vecino invirtiendo el segmento de ruta comprendido entre i y j."""
    candidato = ruta[:]
    candidato[i:j + 1] = reversed(candidato[i:j + 1])
    return candidato
```

```
def hill_climbing(ruta_inicial: list[int], max_rondas: int = 100) -> dict:
    """Hill Climbing de mejor mejora: acepta solo el mejor vecino más barato."""
    actual = ruta_inicial[:]
    costo_actual = costo_ruta(actual)
    historial = [costo_actual]
    evaluaciones = 0
    inicio = time.perf_counter()

    for _ in range(max_rondas):
        mejor_vecino = actual
        mejor_costo = costo_actual
        # El vecindario 2-opt considera todas las inversiones de segmentos.
        for i in range(len(actual) - 1):
            for j in range(i + 1, len(actual)):
                candidato = vecino_2opt(actual, i, j)
                candidato_costo = costo_ruta(candidato)
                evaluaciones += 1
                if candidato_costo < mejor_costo - 1e-9:
                    mejor_vecino = candidato
                    mejor_costo = candidato_costo

        # Si ningún vecino mejora la función, se alcanzó un óptimo local.
        if mejor_costo >= costo_actual - 1e-9:
            historial.append(costo_actual)
            break
        actual = mejor_vecino[:]
        costo_actual = mejor_costo
        historial.append(costo_actual)

    segundos = time.perf_counter() - inicio
    return {
        "ruta": actual,
        "costo": costo_actual,
        "historial": historial,
        "evaluaciones": evaluaciones,
        "segundos": segundos,
        "rondas": len(historial) - 1,
    }


def simulated_annealing(
    ruta_inicial: list[int],
    temperatura_inicial: float = 12.0,
    eta: float = 0.997,
    iteraciones: int = 5000,
    semilla: int = 2026,
) -> dict:
    """Simulated Annealing: permite aceptar empeoramientos según temperatura."""
    generador = random.Random(semilla)
    actual = ruta_inicial[:]
    costo_actual = costo_ruta(actual)
    mejor_ruta = actual[:]
    mejor_costo = costo_actual
    historial = [mejor_costo]
    aceptados = 0
    inicio = time.perf_counter()

    for t in range(iteraciones):
        temperatura = temperatura_inicial * (eta ** t)
        i, j = sorted(generador.sample(range(len(actual)), 2))
        candidato = vecino_2opt(actual, i, j)
        costo_candidato = costo_ruta(candidato)
        diferencia = costo_candidato - costo_actual

        # Mejoras siempre aceptadas; empeoramientos se aceptan con probabilidad
        # exp(-diferencia / temperatura), lo que facilita escapar de óptimos locales.
        aceptar = diferencia < 0
        if not aceptar and temperatura > 1e-12:
            aceptar = generador.random() < math.exp(-diferencia / temperatura)
        if aceptar:
            actual = candidato
            costo_actual = costo_candidato
            aceptados += 1

        if costo_actual < mejor_costo - 1e-9:
            mejor_ruta = actual[:]
            mejor_costo = costo_actual
        historial.append(mejor_costo)
```

```
    segundos = time.perf_counter() - inicio
    return {
        "ruta": mejor_ruta,
        "costo": mejor_costo,
        "historial": historial,
        "evaluaciones": iteraciones,
        "segundos": segundos,
        "iteraciones": iteraciones,
        "aceptados": aceptados,
        "T0": temperatura_inicial,
        "eta": eta,
    }


def buscar_optimo_exacto() -> dict:
    """Enumera las (N-1)! rutas para obtener un punto de referencia exacto."""
    mejor_ruta = None
    mejor_costo = float("inf")
    evaluaciones = 0
    for permutacion in itertools.permutations(range(1, N)):
        costo = costo_ruta(list(permutacion))
        evaluaciones += 1
        if costo < mejor_costo:
            mejor_costo = costo
            mejor_ruta = list(permutacion)
    return {"ruta": mejor_ruta, "costo": mejor_costo, "evaluaciones": evaluaciones}


def nombres_ruta(ruta: list[int]) -> list[str]:
    """Convierte índices a nombres e incluye depósito al inicio y al final."""
    return [NOMBRES[i] for i in [DEPOSITO] + ruta + [DEPOSITO]]


def normalizar_historial(historial: list[float], puntos: int = 101) -> list[float]:
    """Muestrea un historial para comparar visualmente ejecuciones de distinta duración."""
    if len(historial) == 1:
        return historial * puntos
    resultado = []
    for indice in range(puntos):
        posicion = indice * (len(historial) - 1) / (puntos - 1)
        resultado.append(historial[round(posicion)])
    return resultado


def crear_grafica_convergencia(hc: dict, sa: dict, exacto: dict) -> Path:
    """Guarda la gráfica de convergencia en escala de progreso normalizado."""
    x = list(range(101))
    fig, ax = plt.subplots(figsize=(9, 5.2))
    ax.plot(x, normalizar_historial(hc["historial"]), label="Hill Climbing", linewidth=2)
    ax.plot(x, normalizar_historial(sa["historial"]), label="Simulated Annealing", linewidth=1.8)
    ax.axhline(exacto["costo"], linestyle="--", linewidth=1.4,
               label=f"Óptimo exacto: {exacto['costo']:.2f} km")
    ax.set_title("Convergencia de la búsqueda local")
    ax.set_xlabel("Progreso normalizado de cada ejecución (%)")
    ax.set_ylabel("Distancia total de la ruta (km simulados)")
    ax.grid(True, alpha=0.25)
    ax.legend()
    fig.tight_layout()
    destino = FIG_DIR / "convergencia_busqueda_local.png"
    fig.savefig(destino, dpi=180)
    plt.close(fig)
    return destino


def crear_grafica_rutas(hc: dict, sa: dict) -> Path:
    """Dibuja las rutas finales de ambos algoritmos sobre las coordenadas simuladas."""
    grafo = nx.Graph()
    for i, nombre in enumerate(NOMBRES):
        grafo.add_node(i, label=nombre)
    posiciones = {i: COORDENADAS[i] for i in range(N)}

    fig, ejes = plt.subplots(1, 2, figsize=(12, 5.2))
    resultados = [(hc, "Hill Climbing"), (sa, "Simulated Annealing")]
    for ax, (resultado, titulo) in zip(ejes, resultados):
        grafo_ruta = nx.Graph()
        grafo_ruta.add_nodes_from(range(N))
        camino = [DEPOSITO] + resultado["ruta"] + [DEPOSITO]
        grafo_ruta.add_edges_from(zip(camino[:-1], camino[1:]))
        nx.draw_networkx_edges(grafo_ruta, posiciones, ax=ax, width=2, edge_color="#3178A8")
```

```
        nx.draw_networkx_nodes(grafo_ruta, posiciones, ax=ax, node_size=420,
                               node_color="#F4D35E", edgecolors="#333333")
        etiquetas = {i: NOMBRES[i] for i in range(N)}
        nx.draw_networkx_labels(grafo_ruta, posiciones, etiquetas, ax=ax, font_size=7)
        ax.set_title(f"{titulo}\nCosto: {resultado['costo']:.2f} km simulados")
        ax.set_axis_off()
    fig.suptitle("Rutas finales obtenidas por búsqueda local")
    fig.tight_layout()
    destino = FIG_DIR / "rutas_comparadas.png"
    fig.savefig(destino, dpi=180, bbox_inches="tight")
    plt.close(fig)
    return destino


def main() -> None:
    """Ejecuta los métodos, genera figuras y guarda los resultados reproducibles."""
    generador = random.Random(1010)
    ruta_inicial = list(range(1, N))
    generador.shuffle(ruta_inicial)
    costo_inicial = costo_ruta(ruta_inicial)

    hc = hill_climbing(ruta_inicial)
    sa = simulated_annealing(ruta_inicial)
    exacto = buscar_optimo_exacto()
    crear_grafica_convergencia(hc, sa, exacto)
    crear_grafica_rutas(hc, sa)

    resultados = {
        "nota_datos": "Coordenadas y distancias simuladas para fines académicos.",
        "ubicaciones": UBICACIONES,
        "ruta_inicial": nombres_ruta(ruta_inicial),
        "costo_inicial_km": costo_inicial,
        "hill_climbing": {
            "ruta": nombres_ruta(hc["ruta"]),
            "costo_km": hc["costo"],
            "rondas": hc["rondas"],
            "evaluaciones": hc["evaluaciones"],
            "tiempo_segundos": hc["segundos"],
        },
        "simulated_annealing": {
            "ruta": nombres_ruta(sa["ruta"]),
            "costo_km": sa["costo"],
            "iteraciones": sa["iteraciones"],
            "evaluaciones": sa["evaluaciones"],
            "aceptados": sa["aceptados"],
            "temperatura_inicial": sa["T0"],
            "eta": sa["eta"],
            "tiempo_segundos": sa["segundos"],
        },
        "optimo_exacto": {
            "ruta": nombres_ruta(exacto["ruta"]),
            "costo_km": exacto["costo"],
            "rutas_evaluadas": exacto["evaluaciones"],
        },
    }
    salida_json = BASE_DIR / "resultados_2_3.json"
    salida_json.write_text(json.dumps(resultados, indent=2, ensure_ascii=False), encoding="utf-8")

    print("ACTIVIDAD 2.3 - RESULTADOS (distancias simuladas)")
    print(f"Ruta inicial: {resultados['ruta_inicial']}")
    print(f"Costo inicial: {costo_inicial:.2f} km")
    print(f"Hill Climbing: {hc['costo']:.2f} km; rondas={hc['rondas']}; "
          f"evaluaciones={hc['evaluaciones']}; tiempo={hc['segundos']:.6f} s")
    print(f"Simulated Annealing: {sa['costo']:.2f} km; iteraciones={sa['iteraciones']}; "
          f"evaluaciones={sa['evaluaciones']}; aceptados={sa['aceptados']}; "
          f"tiempo={sa['segundos']:.6f} s")
    print(f"Óptimo exacto: {exacto['costo']:.2f} km; rutas enumeradas={exacto['evaluaciones']}")
    print(f"Figuras: {FIG_DIR}")
    print(f"Resultados: {salida_json}")


if __name__ == "__main__":
    main()
```

Declaración de uso de inteligencia artificial

Herramienta: ChatGPT, de OpenAI.

Prompt utilizado: “Implementa Hill Climbing y Simulated Annealing para optimizar una ruta de reparto; incluye planteamiento, función objetivo, vecindad, parámetros, gráfica de convergencia, comparación y complejidad.”

Uso: apoyo para estructurar el planteamiento, formalizar el problema, redactar explicaciones, construir y comentar el código Python y organizar los resultados.

Verificación: el archivo Python fue ejecutado; se revisaron las rutas, los costos, la gráfica de convergencia y la comparación con la enumeración de 40,320 rutas. Las coordenadas y distancias se etiquetaron como simuladas.

Los datos generados para el ejemplo se identifican como simulados. La interpretación final debe ser revisada por el estudiante antes de entregar la actividad.